WOXSEN UNIVERSITY
SCHOOL OF TECHNOLOGY

PROJECT-BASED LEARNING (PBL) REPORT

DATABASE MANAGEMENT SYSTEMS

GYM MEMBERSHIP AND FITNESS SESSION MANAGEMENT SYSTEM

Submitted by: G. Tejaswi

Roll Number: 25WU0102092

ACKNOWLEDGEMENT

I express my sincere gratitude to Woxsen University, School of Technology, for providing the opportunity to undertake this Project-Based Learning activity as part of the Database Management Systems course. I thank the course faculty for guidance and feedback through the review stages. The reviews helped structure the project from requirement analysis and ER modelling to relational design, normalization, SQL operations, application planning and testing. I also acknowledge the use of the prescribed report template and review materials in preparing this document.

UNDERTAKING

I hereby declare that the project entitled “Gym Membership and Fitness Session Management System” is prepared for the Database Management Systems Project-Based Learning activity. The report describes the proposed logical database and application design using the supplied project reviews as the basis. Any implementation status, execution results, screenshots, repository details and personal details that are not evidenced in the supplied material are explicitly left for verification and completion before submission. Sources and technical documentation consulted should be acknowledged in the final submitted version.

Signature: _____

Date: _____

Name: G. Tejaswi

Roll Number: 25WU0102092

TABLE OF CONTENTS

Abstract

Keywords

List of Tables

List of Figures

List of Acronyms

CHAPTER 1 — PROBLEM STATEMENT, SOLUTION STRATEGY AND ER DIAGRAM

1.1 Problem Statement

1.2 Objectives

1.3 Scope and Assumptions

1.4 Proposed Solution Strategy

1.5 Functional Requirements

1.6 Entity-Relationship Diagram (ERD)

1.7 Entity and Relationship Summary

CHAPTER 2 — LOGICAL DATABASE DESIGN

2.1 Relational Schema

2.2 Keys and Integrity Constraints

2.3 Functional Dependencies

2.4 Normalization up to 3NF

2.5 Data Dictionary

2.6 Implementation of Logical Design: DDL and Sample Data

CHAPTER 3 — APPLICATION DEVELOPMENT USING LOGICAL DATABASE DESIGN

3.1 Application Overview

3.2 Development Environment and Database Connectivity

3.3 Application Modules and Database Mapping

3.4 CRUD Operations

3.5 Business Rules, Validation and Error Handling

3.6 Reports and Database Operations

3.7 Application Screens

3.8 Testing and Results

CHAPTER 4 — CONCLUSION

4.1 Limitations and Future Enhancements

CHAPTER 5 — REFERENCES

APPENDIX A — COMPLETE DATABASE SCRIPT

APPENDIX B — APPLICATION SOURCE CODE

APPENDIX C — ADDITIONAL TEST DATA AND OUTPUTS

Page numbers should be updated in Word after final formatting.

LIST OF TABLES

- Table 1. Scope and Assumptions
- Table 2. Proposed Solution Strategy
- Table 3. Functional Requirements
- Table 4. Entity and Relationship Summary
- Table 5. Relational Schema
- Table 6. Keys and Integrity Constraints
- Table 7. Normalization Status up to 3NF
- Table 8. Data Dictionary
- Table 9. Database Implementation Environment
- Table 10. Application Development Environment
- Table 11. Application Module-to-Database Mapping
- Table 12. CRUD Operations
- Table 13. Business Rules, Validation and Error Handling
- Table 14. Reports and Database Operations
- Table 15. Test Cases

LIST OF FIGURES

- Figure 1. Entity-Relationship Diagram of the Proposed System
- Figure 2. Member and Subscription Management Screen (to be captured)
- Figure 3. Class Booking and Schedule Screen (to be captured)
- Figure 4. Attendance and Assessment Screen (to be captured)
- Figure 5. Reports Dashboard (to be captured)

LIST OF ACRONYMS

Acronym	Expansion
CRUD	Create, Read, Update, Delete
DBMS	Database Management System
ERD	Entity-Relationship Diagram
FK	Foreign Key
GUI	Graphical User Interface
PK	Primary Key
SQL	Structured Query Language
3NF	Third Normal Form

ABSTRACT

Gym facilities often maintain member, subscription, trainer, class, attendance, payment and equipment records across paper registers or disconnected spreadsheets. These practices can produce duplicate records, missed subscription expiries, booking conflicts, inaccurate attendance and time-consuming reporting. This project proposes a relational Gym Membership and Fitness Session Management System that centralizes operational data and supports controlled, traceable workflows. The design begins with requirement analysis and an entity-relationship model, then maps the entities into a relational schema with primary keys, foreign keys, uniqueness rules, required fields, range checks and defaults. The logical model is organized to Third Normal Form (3NF) by separating member identity, plan definitions, subscription lifecycle, payment transactions, class schedules, registrations, attendance, trainer records, assessments, workout programmes, equipment and maintenance history. The proposed implementation targets MySQL and a Java or Python desktop interface using parameterized database queries. Planned functions include member and trainer management, subscription renewal, class scheduling and booking, attendance marking, assessment entry, equipment maintenance and reports for active or expired members, attendance, trainer load, renewals and revenue. Transaction checks are required for active membership, class capacity and trainer schedule conflicts. The report distinguishes

proposed design and illustrative data from application execution evidence that must be captured during implementation and testing.

Keywords: Gym Management, Relational Database, Entity-Relationship Diagram, 3NF Normalization, SQL, Transaction Validation

CHAPTER 1

PROBLEM STATEMENT, SOLUTION STRATEGY AND ER DIAGRAM

1.1 Problem Statement

Many fitness centres manage member registrations, membership plans, trainer assignments, fitness sessions, schedules, bookings, attendance, payments and equipment records using registers, spreadsheets or separate applications. As the number of members and sessions grows, these records become difficult to reconcile. Duplicate or inconsistent member details, missed expiry dates, overbooked classes, trainer schedule conflicts, incomplete attendance and delayed payment reports can reduce service quality and create additional administrative work.

The proposed system provides a centralized relational database and a small application through which authorized users can manage these records consistently. The database is designed to preserve relationships and validate data at the point of entry. Booking and renewal workflows must be transaction-aware so that an invalid request does not leave partial records. The supplied review material establishes the requirements and design direction; implementation screenshots and observed test outcomes must be added after the application is run.

1.2 Objectives

- Design an ER model and relational schema covering members, plans, subscriptions, trainers, assessments, workout programmes, classes, schedules, registrations, attendance, payments, renewals, branches, equipment and maintenance.
- Normalize the schema to Third Normal Form and document functional dependencies, primary keys, foreign keys and data constraints.
- Implement database tables and realistic sample records using MySQL-compatible SQL, including integrity constraints and useful indexes.

- Develop a Java or Python interface for membership management, trainer assignment, assessments, class scheduling and booking, attendance, renewals, maintenance and reporting.
- Validate business rules and produce management reports for membership status, class attendance, trainer load, renewals, revenue and maintenance.

1.3 Scope and Assumptions

Item	Description
In Scope	Member and branch records; plan definitions; subscriptions and renewals; payment history; trainer profiles and assignments; fitness classes and schedules; bookings and attendance; assessments and workout programmes; equipment and maintenance; SQL reports and a desktop application.
Out of Scope / Exclusions	Online payment gateway integration, biometric access control, wearable-device integration, native mobile application, multi-currency accounting and production deployment are not part of the current design.
Assumptions	Each member has a unique email address; each subscription references one member and one plan; each class schedule has one assigned trainer; a member can register once per schedule; successful payments alone contribute to revenue; the database server enforces declared constraints.
Implementation status	The supplied Review 1–3 materials include conceptual designs and illustrative screens/data. They do not prove that a working application or all tests have been executed; these results are therefore identified as pending verification.

1.4 Proposed Solution Strategy

The system follows a layered approach: a Java Swing/JavaFX or Python Tkinter presentation layer, an application layer for input validation and business rules, and a MySQL database layer for persistent records. The application communicates with the database using JDBC or a Python database connector and parameterized SQL. Writes that affect multiple records are performed in transactions, with rollback on failure.

Solution Element	Project-Specific Description
Primary Users / Roles	Administrator manages master records and reports; member views membership and sessions, books sessions and checks status; trainer views assigned sessions, marks attendance and maintains fitness records.

Major Application Modules	Members and plans; subscriptions and payments; trainer and assessment; workout programmes; classes and schedules; booking and attendance; equipment and maintenance; reports.
Proposed Workflow	Register member → select plan → create subscription and record payment → assign branch → publish class schedule → verify subscription and capacity → confirm booking → record attendance → generate reports and reminders.
Important Business Rules	End date must not precede start date; assessment values cannot be negative; class capacity must not be exceeded; trainer schedules must not overlap; booking requires an active subscription; renewal dates must be valid; successful payments are used for revenue.
Expected Reports / Outputs	Active and expired members, renewals due, class-wise registrations and attendance, trainer session load, successful payment revenue, and equipment requiring maintenance.

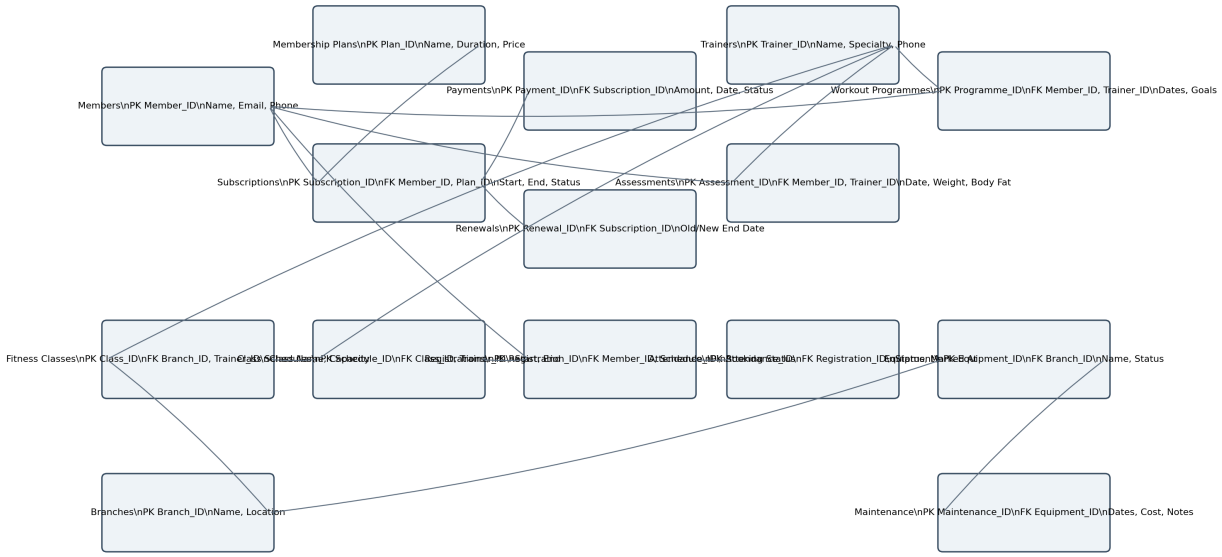
1.5 Functional Requirements

Req. ID	Functional Requirement	Priority
FR-01	Create, view, search and update member and branch records.	High
FR-02	Maintain membership plans and subscription dates/status.	High
FR-03	Maintain trainer profiles, specializations and class assignments.	High
FR-04	Create fitness classes and schedules with valid time ranges.	High
FR-05	Allow booking only when the subscription is active and seats remain.	High
FR-06	Detect trainer schedule overlap before confirming a schedule.	High
FR-07	Record attendance against a valid registration and prevent duplicate attendance.	High
FR-08	Record payments and renewal history; retain failed/pending payment status.	High
FR-09	Record assessments and workout programmes with valid values and dates.	Medium
FR-10	Track equipment status, maintenance dates and maintenance costs.	Medium
FR-11	Generate reports using joins, aggregates, subqueries and views.	High
FR-12	Validate user inputs, handle database errors and rollback failed transactions.	High

1.6 Entity-Relationship Diagram (ERD)

Figure 1 summarizes the major entities and their relationships. The logical schema in Chapter 2 provides the definitive key names and cardinalities. A member can have multiple subscriptions, assessments, workout programmes and registrations; a schedule can have multiple registrations; each registration can have at most one attendance record. A plan can be referenced by many subscriptions, while each subscription references one plan.

Figure 1. Entity-Relationship Diagram of the Proposed System



1.7 Entity and Relationship Summary

Entity / Relationship	Major Attributes / Cardinality	Purpose / Rule
Members	Member_ID; name, email, phone, DOB, join date	Central member identity; email is unique.
Membership Plans	Plan_ID; name, duration, price	Defines price and duration independently from a member.
Subscriptions	Subscription_ID; Member_ID, Plan_ID, dates, status	Member 1:N subscriptions; valid date range required.
Payments / Renewals	Payment_ID and Renewal_ID; subscription references	Stores payment history and old/new expiry dates separately.
Trainers	Trainer_ID; name, specialization, contact	Supports assignments and trainer-load reporting.
Assessments / Workout Programmes	Member_ID and optional/required Trainer_ID	Tracks fitness measurements and personalized programmes.
Fitness Classes / Class Schedules	Class_ID; Schedule_ID; trainer, start/end, capacity	A class has schedules; schedule time range must be valid.
Registrations / Attendance	Registration_ID; unique Member_ID + Schedule_ID	Avoids duplicate booking; attendance is recorded once per registration.
Branches / Equipment / Maintenance	Branch_ID; Equipment_ID; Maintenance_ID	Tracks location, availability, service history and costs.

CHAPTER 2

LOGICAL DATABASE DESIGN

2.1 Relational Schema

The ER model is mapped to the following relations. PK identifies a primary key and FK identifies a foreign key. The full MySQL-compatible DDL and sample inserts are included in Appendix A.

S.No.	Table Name	Purpose / Key Attributes
1	Branches	Branch_ID (PK), Branch_Name, Location, Phone

S.No.	Table Name	Schema / Key Attributes	Purpose
1	Branches	Branch_ID (PK), Branch_Name, Location, Phone	Gym branch master
2	Members	Member_ID (PK), Full_Name, Email (UQ), Phone, Date_of_Birth, Join_Date, Emergency_Contact	Member identity
3	Membership_Plans	Plan_ID (PK), Plan_Name (UQ), Duration_Days, Price, Description	Plan catalogue
4	Trainers	Trainer_ID (PK), Full_Name, Email (UQ), Phone, Specialization, Hire_Date	Trainer master
5	Subscriptions	Subscription_ID (PK), Member_ID (FK), Plan_ID (FK), Start_Date, End_Date, Status	Membership lifecycle
6	Payments	Payment_ID (PK), Subscription_ID (FK), Payment_Date, Amount, Method, Status, Reference_Code (UQ)	Payment transactions
7	Renewals	Renewal_ID (PK), Subscription_ID (FK), Previous_End_Date, New_End_Date, Renewal_Date, Payment_ID (FK)	Renewal audit history
8	Fitness_Classes	Class_ID (PK), Branch_ID (FK), Trainer_ID (FK), Class_Name, Capacity	Class definition
9	Class_Schedules	Schedule_ID (PK), Class_ID (FK), Trainer_ID (FK), Start_Time, End_Time, Room_Name	Dated/time-bound sessions
10	Registrations	Registration_ID (PK), Member_ID (FK), Schedule_ID (FK), Booking_Date, Status	Member booking
11	Attendance	Attendance_ID (PK), Registration_ID (FK/UQ), Attendance_Status, Marked_At, Marked_By_Trainer_ID (FK)	Attendance record
12	Assessments	Assessment_ID (PK), Member_ID (FK), Trainer_ID (FK), Assessment_Date, Weight_Kg, Height_Cm, Body_Fat_Percent, Notes	Fitness measurements

13	Workout_Programmes	Programme_ID (PK), Member_ID (FK), Trainer_ID (FK), Programme_Name, Start_Date, End_Date, Goal	Workout plan history
14	Equipment	Equipment_ID (PK), Branch_ID (FK), Equipment_Name, Serial_Number (UQ), Status, Purchase_Date	Equipment register
15	Maintenance	Maintenance_ID (PK), Equipment_ID (FK), Service_Date, Next_Service_Date, Cost, Notes	Equipment service history

2.2 Keys and Integrity Constraints

Table / Area	Key / Constraint	Attribute(s)	Business Rule Enforced
Members	PK, UNIQUE, NOT NULL	Member_ID; Email; Full_Name, Phone	Identifies a member and prevents duplicate email.
Membership_Plans	PK, UNIQUE, CHECK	Plan_ID; Plan_Name; Duration_Days > 0; Price >= 0	Plan names are distinct; duration and price are valid.
Subscriptions	PK, FK, CHECK, DEFAULT	Member_ID, Plan_ID; Start_Date, End_Date; Status	References valid member/plan; dates ordered; default status provided.
Payments	PK, FK, CHECK, UNIQUE	Subscription_ID; Amount; Status; Reference_Code	Payment is linked to a subscription; amount non-negative.
Renewals	PK, FK, CHECK, UNIQUE	Subscription_ID; New_End_Date; Subscription_ID + New_End_Date	Renewal extends expiry and avoids duplicate renewal date.
Fitness_Classes	PK, FK, CHECK	Branch_ID, Trainer_ID; Capacity	Valid branch/trainer and positive class capacity.
Class_Schedules	PK, FK, CHECK, INDEX	Class_ID, Trainer_ID; Start_Time, End_Time	Valid schedule range; index assists trainer/time checks.
Registrations	PK, FK, UNIQUE, DEFAULT, CHECK	Member_ID, Schedule_ID; Status	One registration per member/ schedule; allowed statuses only.
Attendance	PK, FK, UNIQUE, CHECK	Registration_ID; Attendance_Status	At most one attendance row per registration.
Assessments	PK, FK, CHECK	Member_ID, Trainer_ID; Weight, Height, Body_Fat_Percent	Non-negative measures; body-fat percentage within 0–100.
Workout_Programmes	PK, FK, CHECK	Member_ID, Trainer_ID; Start_Date, End_Date	Programme dates must be ordered.
Equipment / Maintenance	PK, FK, CHECK, DEFAULT	Branch_ID, Equipment_ID; Status, Cost, service dates	Valid equipment status; maintenance cost non-negative.

Application-level rules: active-subscription eligibility, class-capacity enforcement and trainer overlap prevention require transactional logic in addition to basic table constraints. The booking operation should lock the schedule row, check current confirmed registrations, verify the subscription and check conflicting trainer schedules before inserting. If any check fails, the transaction must roll back.

2.3 Functional Dependencies

- Member_ID → Full_Name, Email, Phone, Date_of_Birth, Join_Date, Emergency_Contact
- Email → Member_ID, Full_Name, Phone, Date_of_Birth, Join_Date, Emergency_Contact (candidate-key dependency, assuming email is unique)
- Plan_ID → Plan_Name, Duration_Days, Price, Description
- Trainer_ID → Full_Name, Email, Phone, Specialization, Hire_Date
- Subscription_ID → Member_ID, Plan_ID, Start_Date, End_Date, Status
- Payment_ID → Subscription_ID, Payment_Date, Amount, Method, Status, Reference_Code
- Renewal_ID → Subscription_ID, Previous_End_Date, New_End_Date, Renewal_Date, Payment_ID
- Class_ID → Branch_ID, Trainer_ID, Class_Name, Description, Capacity
- Schedule_ID → Class_ID, Trainer_ID, Start_Time, End_Time, Room_Name
- Registration_ID → Member_ID, Schedule_ID, Booking_Date, Status; (Member_ID, Schedule_ID) → Registration_ID
- Attendance_ID → Registration_ID, Attendance_Status, Marked_At, Marked_By_Trainer_ID
- Assessment_ID → Member_ID, Trainer_ID, Assessment_Date, Weight_Kg, Height_Cm, Body_Fat_Percent, Notes
- Equipment_ID → Branch_ID, Equipment_Name, Serial_Number, Status, Purchase_Date
- Maintenance_ID → Equipment_ID, Service_Date, Next_Service_Date, Cost, Notes

2.4 Normalization up to Third Normal Form (3NF)

Normalization reduces duplicated facts and the insertion, update and deletion anomalies that occur when unrelated facts are stored in one large table. The design is evaluated as follows.

Normal Form	Requirement	Application to This Design

UNF	May contain repeating groups or mixed facts.	A single register combining member, plan, trainer, booking and payment information would repeat values and mix multiple events.
1NF	Each field contains one atomic value and each row is identifiable.	Attributes are scalar; repeating bookings, payments and attendance are represented as separate rows in their own relations.
2NF	1NF plus no non-key attribute depends on only part of a composite candidate key.	Operational tables use single-column surrogate primary keys; the registration alternate key (Member_ID, Schedule_ID) identifies the complete registration row.
3NF	2NF plus no non-key attribute depends transitively on a key.	Plan pricing stays in Membership_Plans, member contact details in Members, trainer details in Trainers, and payment/maintenance history in their own tables.
BCNF note	Every determinant of a non-trivial dependency is a superkey.	The listed dependencies are designed around declared candidate keys. BCNF is not a separate requirement for this project; the target is 3NF.

For example, storing Plan_Name and Price inside every member record would repeat plan facts and could cause update anomalies. Separating Membership_Plans and referencing Plan_ID from Subscriptions keeps plan data consistent while allowing members to hold subscriptions over time.

2.5 Data Dictionary

Table	Attribute	Data Type	Key / Constraint	Description
Members	Member_ID	INT AUTO_INCREMENT	PK	Unique member identifier
Members	Full_Name	VARCHAR(100)	NOT NULL	Member full name
Members	Email	VARCHAR(120)	UNIQUE, NOT NULL	Contact email and candidate key
Members	Phone	VARCHAR(20)	NOT NULL	Contact number
Members	Date_of_Birth	DATE	NULL allowed	Date of birth
Members	Join_Date	DATE	NOT NULL, DEFAULT CURRENT_DATE	Registration date
Membership_Plans	Plan_ID	INT AUTO_INCREMENT	PK	Unique plan identifier
Membership_Plans	Plan_Name	VARCHAR(50)	UNIQUE, NOT NULL	Plan label
Membership_Plans	Duration_Days	INT	CHECK > 0	Plan duration
Membership_Plans	Price	DECIMAL(10,2)	CHECK >= 0	Plan fee
Subscriptions	Subscription_ID	INT AUTO_INCREMENT	PK	Subscription identifier
Subscriptions	Member_ID	INT	FK, NOT NULL	Member holding subscription
Subscriptions	Plan_ID	INT	FK, NOT NULL	Selected plan
Subscriptions	Start_Date / End_Date	DATE	NOT NULL; End >= Start	Validity dates
Subscriptions	Status	VARCHAR(15)	DEFAULT ACTIVE, CHECK	Lifecycle status
Trainers	Trainer_ID	INT AUTO_INCREMENT	PK	Unique trainer identifier
Trainers	Specialization	VARCHAR(100)	NOT NULL	Training specialty
Payments	Payment_ID	INT AUTO_INCREMENT	PK	Payment identifier
Payments	Amount	DECIMAL(10,2)	CHECK >= 0	Payment amount
Payments	Status	VARCHAR(15)	DEFAULT PENDING, CHECK	Payment outcome
Renewals	Renewal_ID	INT AUTO_INCREMENT	PK	Renewal history identifier
Renewals	New_End_Date	DATE	CHECK > Previous_End_Date	Renewed expiry
Fitness_Classes	Class_ID	INT AUTO_INCREMENT	PK	Class identifier
Fitness_Classes	Capacity	INT	CHECK > 0	Maximum class registrations
Class_Schedules	Schedule_ID	INT AUTO_INCREMENT	PK	Scheduled occurrence identifier
Class_Schedules	Start_Time / End_Time	DATETIME	CHECK End > Start	Schedule interval
Registrations	Registration_ID	INT AUTO_INCREMENT	PK	Booking identifier
Registrations	Member_ID / Schedule_ID	INT	FK; UNIQUE pair	One booking per member per schedule
Attendance	Registration_ID	INT	FK, UNIQUE	Registration attendance link
Assessments	Weight_Kg / Height_Cm	DECIMAL(5,2)	CHECK >= 0	Fitness measurement values
Assessments	Body_Fat_Percent	DECIMAL(5,2)	CHECK 0–100	Body-fat percentage
Workout_Programmes	Start_Date / End_Date	DATE	CHECK End >= Start	Programme validity
Branches	Branch_ID	INT AUTO_INCREMENT	PK	Branch identifier
Equipment	Equipment_ID	INT AUTO_INCREMENT	PK	Equipment identifier
Equipment	Status	VARCHAR(20)	DEFAULT AVAILABLE, CHECK	Current equipment status
Maintenance	Cost	DECIMAL(10,2)	DEFAULT 0, CHECK >= 0	Service cost
Maintenance	Next_Service_Date	DATE	CHECK >= Service_Date or NULL	Planned next service

2.6 Implementation of Logical Design: DDL and Sample Data

Item	Details
DBMS	MySQL 8.0+ target; SQL script is provided in Appendix A.
DBMS Version	Target version specified for compatibility; actual installed version must be recorded after execution.
Number of Tables	15 base tables.
Sample Data	Illustrative records for two branches, four members, three plans, two trainers, subscriptions, payments, classes, schedules, bookings, attendance, assessments, workout programmes, equipment and maintenance.
Execution Evidence	The script is prepared but the supplied reviews do not include database execution logs. Run the script in the selected DBMS and replace this note with observed output before submission.

Representative DDL

```
CREATE TABLE Membership_Plans (
  Plan_ID INT PRIMARY KEY AUTO_INCREMENT,
  Plan_Name VARCHAR(50) NOT NULL UNIQUE,
  Duration_Days INT NOT NULL CHECK (Duration_Days > 0),
  Price DECIMAL(10,2) NOT NULL CHECK (Price >= 0),
  Description VARCHAR(255)
);
```

The complete DDL, sample INSERT statements, reporting view and example queries are included in Appendix A and the companion SQL file.

CHAPTER 3

APPLICATION DEVELOPMENT USING LOGICAL DATABASE DESIGN

3.1 Application Overview

The planned application is a small desktop client that provides role-appropriate access to the centralized gym database. The administrator maintains members, plans, trainers, branches, classes, schedules and equipment; members can search available classes, view subscription status and request bookings; trainers can view assigned sessions, mark attendance and maintain assessment or workout-programme records. Database queries rather than hard-coded lists should populate the screens. Since the supplied review materials show illustrative interface concepts rather than verified application screenshots, genuine screenshots must be captured after implementation.

3.2 Development Environment and Database Connectivity

Item	Details
Programming Language	Java or Python, as permitted by the project brief; choose one for the actual implementation.
DBMS	MySQL 8.0+ target. PostgreSQL may be substituted if the SQL is adapted and tested.
IDE / Framework / GUI Toolkit	Java: IntelliJ IDEA/Eclipse with Swing or JavaFX; Python: VS Code/ PyCharm with Tkinter.
Database Connector / Driver	Java Database Connectivity (JDBC) for Java or a supported MySQL connector for Python.
Operating Environment	Desktop environment with database server access; record the actual OS and versions used.
Repository Link	Not supplied in review documents; add the actual repository URL or source folder reference.

A typical connection lifecycle is: open connection → execute parameterized SQL → fetch and validate results → commit successful writes or roll back failed transactions → close cursor and

connection. Credentials should be stored outside source code, and database exceptions should be logged without exposing passwords or sensitive details.

3.3 Application Modules and Database Mapping

Module / Screen	Purpose	Tables Used	Main Operations
Member Management	Register and maintain members	Members, Branches	Create, search, update
Plans and Subscriptions	Assign plan and monitor expiry	Membership_Plans, Subscriptions, Renewals	Read, create, renew, update status
Payments	Record payment attempts and outcomes	Payments, Subscriptions	Create, search, reconcile
Trainer and Fitness Records	Maintain trainers, assessments and programmes	Trainers, Assessments, Workout_Programmes	Create, read, update
Class Scheduling	Create classes and schedule occurrences	Fitness_Classes, Class_Schedules, Trainers	Create, read, check overlap
Booking	Register members for sessions	Subscriptions, Registrations, Class_Schedules	Validate and create transactionally
Attendance	Record present/absent/late status	Attendance, Registrations	Create, read, prevent duplicate
Equipment Maintenance	Track availability and service dates	Equipment, Maintenance, Branches	Create, update status, report due service
Reports Dashboard	Management summaries	Most operational tables / views	Search, joins, aggregates, subqueries

3.4 CRUD Operations

Operation	Application Feature / Screen	Table(s) Affected	Evidence Required
Create	Add member or create schedule	Members; Fitness_Classes; Class_Schedules	Capture successful form submission and resulting row.
Read / Search	Search member, plan or upcoming class	Members; Subscriptions; Fitness_Classes; Class_Schedules	Capture search criteria and returned records.
Update	Renew subscription or change equipment status	Subscriptions; Renewals; Payments; Equipment	Capture before-and-after values.
Delete / Controlled Removal	Remove an unused draft record where no dependent data exists	Applicable master table	Capture confirmation and referential-integrity behaviour; prefer cancellation/status change for historical transactions.

3.5 Business Rules, Validation and Error Handling

Rule / Validation	How Implemented	Evidence / Message
Unique member email	UNIQUE constraint plus friendly duplicate-key handling.	Duplicate email should be rejected; capture observed message.
Active subscription required	Before booking, check subscription dates/status against the booking date.	Reject booking if no active subscription exists.
Class capacity	Within a transaction, lock schedule row and count confirmed registrations before insert.	Reject when confirmed count equals capacity.
No trainer overlap	Check whether any existing interval overlaps the proposed interval for the same trainer.	Reject conflicting schedule and show the conflicting slot.
Valid dates	CHECK constraints and application date validation for subscriptions, schedules, renewals and programmes.	Reject end date earlier than start date.
Assessment values	Non-negative checks for weight/height; body fat between 0 and 100.	Reject negative or out-of-range measurement.
Payment status	Keep SUCCESS, FAILED, PENDING and REFUNDED distinct; revenue includes SUCCESS only.	Failed/pending payments must not be reported as revenue.
SQL safety	Parameterized queries; avoid string concatenation for user-entered values.	Input containing quotes should be handled as data, not executable SQL.
Transaction failures	Commit only when all checks pass; rollback on exceptions.	No partial booking or renewal record after a failed operation.

3.6 Reports and Database Operations

Report / Operation	SQL / Database Concept Used	Application Output
Active and expired members	JOIN, CASE, date comparison, view	Member, plan, start/end date and status.
Class attendance	LEFT JOIN, COUNT, SUM, GROUP BY	Registrations and attendance by class schedule.
Trainer load	LEFT JOIN, COUNT, GROUP BY	Scheduled sessions per trainer.
Revenue summary	SUM with WHERE Status = SUCCESS	Total confirmed revenue; exclude failed/pending payments.
Renewals due	Date filter over next 30 days and JOIN	Members whose subscription ends soon.
Equipment maintenance	JOIN and date-window filter	Equipment with next service due soon.
Above-average payers	GROUP BY, HAVING and nested AVG subquery	Members whose successful payment total exceeds the average payment amount.
Views	CREATE VIEW for subscription status	Reusable status query for dashboard and reports.
Indexes	Index on trainer and schedule time; PK/FK and unique indexes	Faster lookups for common schedule and identity searches.

The companion SQL script includes representative SQL for the reports above. The outputs should be executed against the populated database and captured; this report does not claim unobserved query results.

3.7 Application Screens

The review materials contain illustrative interface placeholders, not evidence of a completed executable application. Insert actual screenshots after implementation. At minimum, capture a member form with data-entry validation, a member/search results screen, a booking or renewal transaction, an attendance/assessment screen, and a report output.

[Figure 2. Member and Subscription Management Screen (insert genuine screenshot)]

[Figure 3. Class Scheduling and Booking Screen (insert genuine screenshot)]

[Figure 4. Attendance and Assessment Screen (insert genuine screenshot)]

[Figure 5. Reports Dashboard (insert genuine screenshot)]

3.8 Testing and Results

The following table is a test plan, not a claim of completed execution. The “Actual Result” and “Status” columns are intentionally marked pending because the review files do not include execution logs. Update them with observed outcomes after running the database and application.

Test ID	Feature	Input / Action	Expected Result	Actual Result	Status
TC-01	Member email uniqueness	Insert a second member with an existing email.	Insert rejected by UNIQUE constraint.	Pending execution	Not run
TC-02	Foreign-key integrity	Insert a subscription with a nonexistent Member_ID.	Insert rejected by foreign key.	Pending execution	Not run
TC-03	Date validation	Create subscription with End_Date before Start_Date.	Insert rejected by validation/ CHECK.	Pending execution	Not run
TC-04	Assessment validation	Enter negative weight or body-fat above 100.	Invalid assessment rejected.	Pending execution	Not run
TC-05	Active subscription booking	Book a class using an expired subscription.	Booking rejected; no registration saved.	Pending execution	Not run
TC-06	Capacity limit	Attempt booking when confirmed registrations equal capacity.	New booking rejected; count remains within capacity.	Pending execution	Not run
TC-07	Trainer overlap	Create overlapping schedules for the same trainer.	Conflicting schedule rejected.	Pending execution	Not run
TC-08	Duplicate booking	Register the same member twice for one schedule.	UNIQUE pair prevents duplicate registration.	Pending execution	Not run
TC-09	Revenue accuracy	Compare SUCCESS payments with FAILED/ PENDING payments.	Revenue includes only SUCCESS amounts.	Pending execution	Not run
TC-10	Transaction rollback	Force a booking validation failure after transaction begins.	No partial registration remains after rollback.	Pending execution	Not run

Result interpretation: once executed, the tests should demonstrate both positive flows (valid member, valid booking, valid assessment and reports) and negative flows (duplicate email, invalid dates, capacity overflow, trainer overlap and invalid measurements). Report the actual DBMS version, test date, observed output and any defects fixed. Until then, the application and test status remains unverified.

CHAPTER 4

CONCLUSION

The Gym Membership and Fitness Session Management System provides a structured design for managing member identity, membership plans and subscriptions, trainers, assessments, workout programmes, fitness classes, schedules, registrations, attendance, payments, renewals, branches, equipment and maintenance. The logical model separates independently maintained facts into related tables and documents primary keys, foreign keys, candidate keys, constraints and functional dependencies. This organization supports the target of Third Normal Form and reduces common insertion, update and deletion anomalies.

The SQL appendix provides a MySQL-oriented implementation baseline, sample records, a status view and report queries using joins, aggregates, subqueries and date filters. The application design connects these relations to a role-aware desktop interface and defines transaction checks for active subscriptions, class capacity and trainer schedule conflicts. The project's final demonstration should include actual application screenshots and observed positive and negative test results; those items are not evidenced by the supplied review files and must be completed before submission.

4.1 Limitations and Future Enhancements

- The supplied reviews do not include a verified running application, database execution logs or genuine application screenshots; implementation and testing evidence must be added.
- Payment processing is represented as recorded transactions; integration with a payment gateway and reconciliation workflow can be added later.
- Role-based access control is described at the application level; production deployment should add authenticated accounts, permissions, audit logs and secure credential management.
- Future improvements may include automated expiry reminders, waitlists, recurring schedule support, multi-branch analytics, online booking and mobile access.

- Concurrency handling for capacity and trainer conflicts must be implemented and stress-tested in the selected DBMS; basic CHECK constraints alone do not enforce cross-row rules.

CHAPTER 5

REFERENCES

The following technical documentation is relevant to the proposed design. Verify access dates and cite these sources in the final report wherever their concepts are discussed.

[1] Oracle, “MySQL 8.0 Reference Manual,” MySQL Documentation. [Online]. Available: https://dev.mysql.com/doc/refman/8.0/en/

[2] Oracle, “MySQL Tutorial,” MySQL Documentation. [Online]. Available: https://dev.mysql.com/doc/refman/8.0/en/tutorial.html

[3] Oracle, “MySQL InnoDB Locking and Transaction Model,” MySQL Documentation. [Online]. Available: https://dev.mysql.com/doc/refman/8.0/en/innodb-transaction-model.html

[4] Oracle, “JDBC Basics,” Java Tutorials. [Online]. Available: https://docs.oracle.com/javase/tutorial/jdbc/basics/

[5] Python Software Foundation, “sqlite3 — DB-API 2.0 interface for SQLite databases,” Python Documentation. [Online]. Available: https://docs.python.org/3/library/sqlite3.html

Note: Record the actual date accessed and ensure every listed source was consulted. If Python is implemented with MySQL rather than SQLite, cite the specific connector documentation used.

APPENDIX A — COMPLETE DATABASE SCRIPT

The complete SQL script is supplied as a separate companion file named "Appendix_A_Gym_DBMS.sql". It includes table definitions, keys and constraints, sample data, a reporting view, example joins, aggregate queries, a nested query, and a capacity-check query. Run it in a test database and confirm successful execution before submission.

APPENDIX B — APPLICATION SOURCE CODE

Source code / repository reference: [Insert the actual repository URL or submitted source folder]. The implementation should use parameterized queries, controlled transaction boundaries, exception handling and database connections that are closed reliably.

APPENDIX C — ADDITIONAL TEST DATA AND OUTPUTS

Add genuine screenshots or exported query results after executing the database and application. Include: (i) successful member creation and search, (ii) a rejected invalid input, (iii) booking or renewal transaction, (iv) attendance entry, and (v) at least one report. Do not submit illustrative mock screens as if they were execution evidence.